

Investigating Real-Time Communication Between Asymmetric Processors with OpenAMP

Modern mixed-criticality embedded systems increasingly rely on **Asymmetric Multiprocessing (AMP)** architectures, such as the Xilinx Zynq UltraScale+ MPSoC, which combine high-performance application processors (e.g., ARM Cortex-A53) with deterministic real-time processing units (e.g., ARM Cortex-R5). While the OpenAMP framework traditionally facilitates **Inter-Processor Communication (IPC)** across these heterogeneous cores using the RPMsg protocol over shared memory, this project investigates an emerging, experimental paradigm known as "**hypervisorless VirtIO**" (or AMP VirtIO).

This novel approach eliminates the need for a heavyweight hypervisor. Instead, it utilizes a lightweight user-space **Physical Machine Monitor (PMM)** on the Linux master core to expose standard VirtIO devices, such as network (`virt-net`) and entropy generation (`virt-rng`), directly to the Zephyr RTOS running on the remote core. The communication relies on a shared memory region and Inter-Processor Interrupts (IPIs), which replace traditional hypervisor VM-exits.

Through architectural mapping, source-code analysis, and hands-on experimentation within a QEMU-emulated environment, the aim of this work is to evaluate the hypervisorless VirtIO architecture. The final synthesis compares its real-time capabilities, latency overheads, and security limitations against classic RPMsg, while also contextualizing its role in the future of industrial and automotive computing, such as the evolution towards the `virtio-msg` protocol and the Linaro HVAC project.

1 System Architecture and Execution Flow

The first step will be mapping the architecture and detailing the execution flow of the hypervisorless VirtIO demonstration (`demo4`), a containered script that involves operating within a QEMU-emulated MPSoC environment.

1.1 System Architecture Diagram

This is a visual map of the overall system from a top-level perspective.

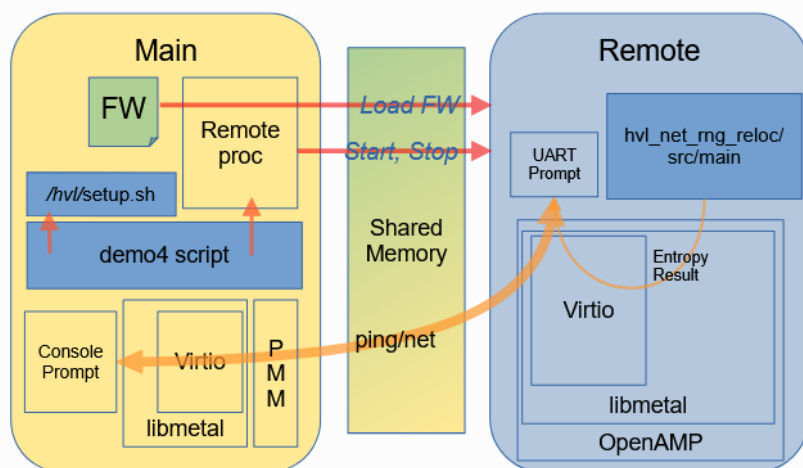


Figure 1: Overview of the system architecture

The overall architecture of the hypervisorless VirtIO demonstration is deployed on a heterogeneous asymmetric multiprocessing (AMP) System-on-a-Chip (SoC), specifically emulating the Zynq UltraScale+ MPSoC. As illustrated in [Figure 1](#), the system is logically divided into two main processing domains. The **Main** domain (Master) runs a Linux operating system on an ARM Cortex-A53 application processor. The **Remote** domain runs the Zephyr Real-Time Operating System (RTOS) on an ARM Cortex-R5F real-time processor, executing the `hvl_net_rng_reloc` firmware application.

Unlike traditional VirtIO implementations that rely on a heavyweight hypervisor (such as KVM) to manage virtual machines and trap hardware accesses via VM-exits, this architecture is completely **hypervisorless**. Instead, the Linux master utilizes a lightweight, user-space Physical Machine Monitor (PMM) instantiated via the `lkvm` tool. The PMM exposes the standard VirtIO backend devices, specifically a network interface and a random number generator (RNG), directly to the Zephyr RTOS. The communication between the two completely independent software stacks is bridged by the OpenAMP framework and the `libmetal` library, leveraging a central shared memory region.

To take advantage of the possibility of using lateral communication without a hypervisor, the system exploits the VirtIO-MMIO (Memory-Mapped I/O) transport mechanism over a statically defined physical shared memory region, located at the base address `0x37000000`.

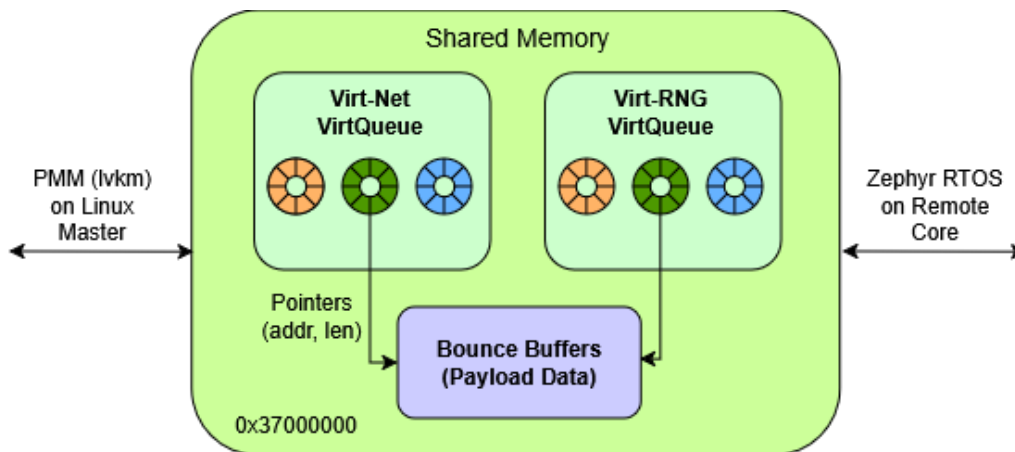


Figure 2: Shared Memory detail

As detailed in [Figure 2](#), both the Linux PMM and the Zephyr RTOS possess symmetric read and write access to this memory space.

Within this shared memory, two primary VirtQueues are instantiated: **Virt-Net** for network traffic and **Virt-RNG** for entropy generation. A crucial architectural adaptation of this hypervisorless approach is the explicit separation of control paths and data payloads. While the control mechanism relies on the standard three-VRing structure (**Descriptor**, **Available**, and **Used Rings**), the data transport deviates from the KVM paradigm. Here, the Descriptor Ring points to pre-allocated, contiguous **Bounce Buffers** for data exchange.

1.2 Component Description

To fully grasp the execution environment of this hypervisorless architecture, it is essential to analyze the individual software and hardware components that functionally replace the traditional virtualized stack.

- **The Physical Machine Monitor (PMM):** in an Asymmetric Multiprocessing (AMP) paradigm, the need for a Virtual Machine Monitor (VMM) is clearly absent. Hence, this term is a misnomer when there is

no virtualization, so PMM (Physical Machine Monitor) is used instead. What happens is that the `lkvm` tool is stripped of its CPU virtualization features, operating purely as a user-space PMM. Its primary responsibility is lifecycle management and resource provisioning: it handles the initial setup of the shared memory mapping and instantiates the backend side of the VirtIO protocol, tricking the remote core into seeing native devices without hypervisor mediation.

- **VirtIO-MMIO and statically mapped memory:** the communication utilizes the Memory-Mapped I/O (MMIO) transport. Unlike traditional VirtIO where the driver allocates memory, the PMM statically maps the devices to a pre-defined physical shared memory region. In this demonstration, the `lkvm` process maps the `virt-net` and `virt-rng` devices starting at the physical address `0x37000000`, after a brief `4KB` header.
- **Bounce Buffers:** operating without a hypervisor implies that neither OS can dynamically translate guest-physical addresses into host-virtual addresses. To overcome this limitation, a software "*shim layer*" manifests as pre-allocated, fixed-size **bounce buffers*** within the shared memory. This architecture guarantees that payload data is always written to a mutually accessible physical location. While this introduces an explicit *memory-to-memory* copy operation, it ensures absolute memory alignment and prevents race conditions between the Linux master and the Zephyr remote.
- **Hardware Notifications (IPIs):** without a hypervisor, the system cannot rely on VM-exits to wake up the host. Instead, this architecture uses physical hardware notifications. When the Zephyr driver or the Linux device places a message in the bounce buffers and updates the control rings, it triggers an **Inter-Processor Interrupt (IPI)** to notify the other core. On the Linux side, this is handled by the dedicated `user-mbox.ko` kernel module.

1.3 Execution Flow

The chronological execution flow of the system, from the initial infrastructure setup to a successful bidirectional network ping is shown in [Figure 3](#).

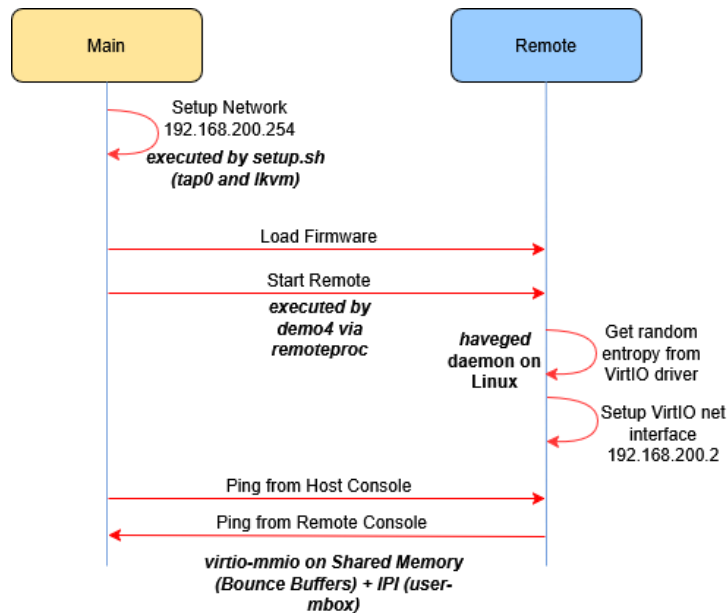


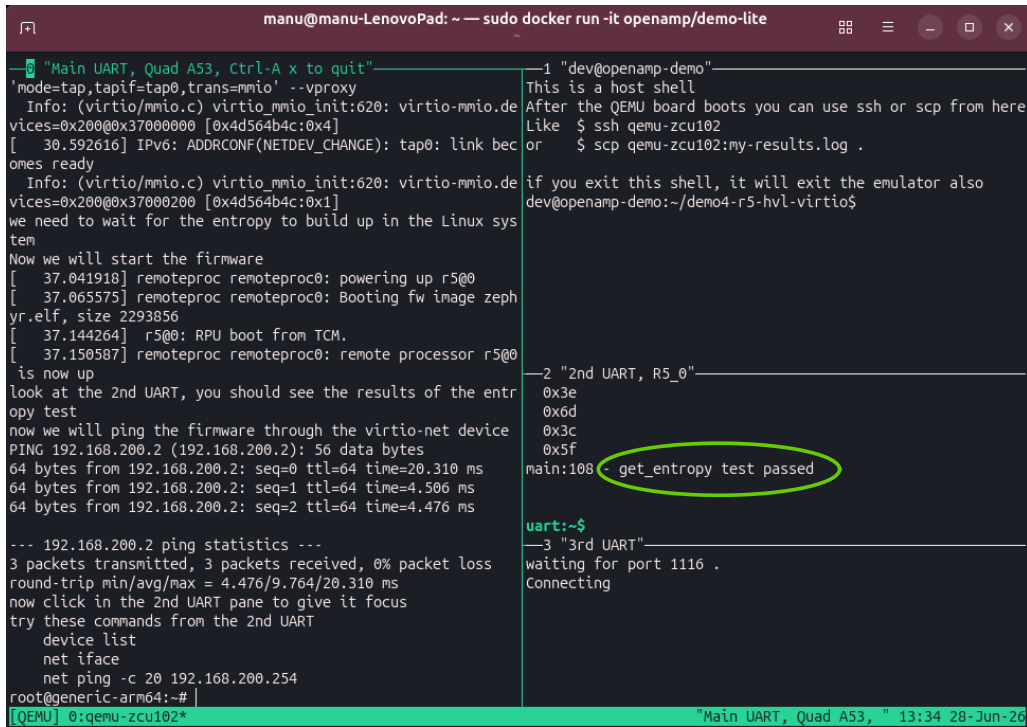
Figure 3: Control Flow

The communication is orchestrated primarily by the Linux Master. As mapped in the sequence diagram, the initialization begins with the `/hvl/setup.sh` script, which operates in the background to configure the virtual network interface (`tap0` at IP `192.168.200.254`) and launch the `lkvm` PMM.

* the word *bounce* suggests that these buffer act as a middle area that allows data to ricochet between the two memory domains divided by a software layer

Simultaneously, the `haveged` daemon begins generating and accumulating random entropy on the Linux side.

Once the infrastructure is ready, the `demo4` script leverages the Linux `remoteproc` sysfs interface to load the Zephyr firmware into memory and issue the start command, releasing the Cortex-R5F core from reset. Upon booting, the Zephyr RTOS immediately accesses the `Virt-RNG` driver to retrieve the accumulated random entropy from the shared memory.



```
manu@manu-LenovoPad: ~ -- sudo docker run -it openamp/demo-lite

--1 "Main UART, Quad A53, Ctrl-A x to quit"
mode=tap,tapif=tap0,trans=mmio' --vproxy
Info: (virtio/mmio.c) virtio_mmio_init:620: virtio-mmio.de
vices=0x200@0x37000000 [0x4d564b4c:0x4]
[ 30.592616] IPv6: ADDRCONF(NETDEV_CHANGE): tap0: link bec
omes ready
Info: (virtio/mmio.c) virtio_mmio_init:620: virtio-mmio.de
vices=0x200@0x37000000 [0x4d564b4c:0x1]
we need to wait for the entropy to build up in the Linux sys
tem
Now we will start the firmware
[ 37.041918] remoteproc remoteproc0: powering up r5@0
[ 37.065575] remoteproc remoteproc0: Booting fw image zeph
yr.elf, size 2293856
[ 37.144264] r5@0: RPU boot from TCM.
[ 37.150587] remoteproc remoteproc0: remote processor r5@0
is now up
look at the 2nd UART, you should see the results of the entr
opy test
now we will ping the firmware through the virtio-net device
PING 192.168.200.2 (192.168.200.2): 56 data bytes
64 bytes from 192.168.200.2: seq=0 ttl=64 time=20.310 ms
64 bytes from 192.168.200.2: seq=1 ttl=64 time=4.506 ms
64 bytes from 192.168.200.2: seq=2 ttl=64 time=4.476 ms

--- 192.168.200.2 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 4.476/9.764/20.310 ms
now click in the 2nd UART pane to give it focus
try these commands from the 2nd UART
device list
net iface
net ping -c 20 192.168.200.254
root@generic-arm64:~#

[QEMU] 0:qemu-zcu102*

--1 "dev@openamp-demo"
This is a host shell
After the QEMU board boots you can use ssh or scp from here
Like $ ssh qemu-zcu102
or $ scp qemu-zcu102:my-results.log .
if you exit this shell, it will exit the emulator also
dev@openamp-demo:~/demo4-r5-hvl-virtio$

--2 "2nd UART, R5_0"
0x3e
0x6d
0x3c
0x5f
main:108 - get_entropy test passed

uart:~$

--3 "3rd UART"
waiting for port 1116 .
Connecting
```

Figure 4: tmux shell showing entropy boot sequence

Finally, when a ping command is executed from the Linux Host Console towards the Zephyr Remote Console (IP `192.168.200.2`), the ICMP packets *do not* travel over a physical network. Instead, they are placed into the shared memory bounce buffers via the `virtio-mmio` transport and an IPI is raised to notify Zephyr through the `user-mbox` driver, successfully fulfilling the hypervisorless paradigm.

```
manu@manu-LenovoPad: ~ — sudo docker run -it open...
— "2nd UART, RS_0"
uart:~$ net ping -c 20 192.168.200.254
net ping -c 20 192.168.200.254
PING 192.168.200.254
28 bytes from 192.168.200.254 to 192.168.200.2: icmp_seq=0 ttl=64 time=366 ms
28 bytes from 192.168.200.254 to 192.168.200.2: icmp_seq=1 ttl=64 time=176 ms
28 bytes from 192.168.200.254 to 192.168.200.2: icmp_seq=2 ttl=64 time=188 ms
28 bytes from 192.168.200.254 to 192.168.200.2: icmp_seq=3 ttl=64 time=256 ms
28 bytes from 192.168.200.254 to 192.168.200.2: icmp_seq=4 ttl=64 time=250 ms
28 bytes from 192.168.200.254 to 192.168.200.2: icmp_seq=5 ttl=64 time=249 ms
28 bytes from 192.168.200.254 to 192.168.200.2: icmp_seq=6 ttl=64 time=270 ms
28 bytes from 192.168.200.254 to 192.168.200.2: icmp_seq=7 ttl=64 time=246 ms
28 bytes from 192.168.200.254 to 192.168.200.2: icmp_seq=8 ttl=64 time=230 ms
28 bytes from 192.168.200.254 to 192.168.200.2: icmp_seq=9 ttl=64 time=234 ms
28 bytes from 192.168.200.254 to 192.168.200.2: icmp_seq=10 ttl=64 time=239 ms
28 bytes from 192.168.200.254 to 192.168.200.2: icmp_seq=11 ttl=64 time=220 ms
28 bytes from 192.168.200.254 to 192.168.200.2: icmp_seq=12 ttl=64 time=244 ms
28 bytes from 192.168.200.254 to 192.168.200.2: icmp_seq=13 ttl=64 time=210 ms
28 bytes from 192.168.200.254 to 192.168.200.2: icmp_seq=14 ttl=64 time=237 ms
28 bytes from 192.168.200.254 to 192.168.200.2: icmp_seq=15 ttl=64 time=245 ms
28 bytes from 192.168.200.254 to 192.168.200.2: icmp_seq=16 ttl=64 time=281 ms
28 bytes from 192.168.200.254 to 192.168.200.2: icmp_seq=17 ttl=64 time=253 ms
28 bytes from 192.168.200.254 to 192.168.200.2: icmp_seq=18 ttl=64 time=252 ms
28 bytes from 192.168.200.254 to 192.168.200.2: icmp_seq=19 ttl=64 time=228 ms
uart:~$
```

Figure 5: Initiating a reverse ping

To display bidirectional communication, a ping from remote is performed, as shown in [Figure 5](#).

2 Code-Level Analysis

The transition from the high-level architectural overview to the actual C source code implementation of the Zephyr RTOS firmware is a crucial step to deeply understand the system in its entirety. The analysis is realized by accessing the `hvl_net_rng_reloc` sample located in the experimental `virtio-exp` branch of the `zephyr-openamp-staging` repository, which allows to map the theoretical hypervisorless concepts to tangible code artifacts.

2.1 Mapping Architecture to C functions

In a traditional virtualized environment, VirtIO devices are dynamically managed by a hypervisor (such as KVM), which allocates VirtQueues in the guest RAM and handles feature negotiation. To establish a hypervisorless VirtIO connection, the Zephyr firmware must be explicitly configured to bypass this dynamic assignment and forcefully map the VirtIO-MMIO transport **directly onto the physical shared memory**.

The transition to this architecture is achieved through specific configuration flags and low-level "glue code". The first piece of this adaptation is found in the project Zephyr configuration file, `prj.conf`.

```

C: > Users > emanu > Downloads > prj.conf
1  CONFIG_ASSERT=y
2  CONFIG_VIRTIO=y
3  CONFIG_VIRTIO_MMIO=y
4  CONFIG_VIRTIO_NETWORK=y
5  CONFIG_VIRTIO_RNG=y
6  CONFIG_ENTROPY_GENERATOR=y
7
8  CONFIG_EARLY_CONSOLE=y
9  CONFIG_SERIAL_INIT_PRIORITY=45
10 CONFIG_CONSOLE_INIT_PRIORITY=46
11
12 CONFIG_SHELL=y
13 CONFIG_NET_SHELL=y
14
15 CONFIG_NET_L2_ETHERNET=y
16 CONFIG_NET_DRIVERS=y
17 CONFIG_NETWORKING=y
18 CONFIG_NET_IPV6=n
19 CONFIG_NET_IPV4=y
20 CONFIG_NET_CONFIG_NEED_IPV4=y
21 CONFIG_NET_DHCPV4=y
22
23 CONFIG_COMPILER_OPT=" -DHVL_VIRTIO"
24 CONFIG_HAVE_CUSTOM_LINKER_SCRIPT=y
25 CONFIG_CUSTOM_LINKER_SCRIPT="linker_r5_hvl.ld"

```

Figure 6: Section of prj.conf

Here, the VirtIO-MMIO transport is explicitly enabled via `CONFIG_VIRTIO_MMIO=y`. More importantly, this file injects a custom compiler flag (`CONFIG_COMPILER_OPT=" -DHVL_VIRTIO"`) and enforces the use of a **custom linker script** (`CONFIG_HAVE_CUSTOM_LINKER_SCRIPT=y`). The `-DHVL_VIRTIO` macro acts as a switch within the OpenAMP libraries to trigger the **hypervisorless-specific routines** instead of the standard virtualized ones.

The most critical architectural adaptation, which answers exactly **where the VirtQueues are located**, resides in the custom linker script `linker_r5_hvl.ld`. Because there is no hypervisor to translate guest-physical addresses to host-virtual addresses, the firmware must be compiled with hardcoded knowledge of the physical memory layout.

The linker script explicitly defines the shared memory base address, perfectly matching the parameters passed to the lkvm PMM in the Linux setup phase:

```

13  #define SHMEM_START_ADDR 0x37000000
14  #define SRAM2_ADDR      0x38000000
15  #define SRAM2_SIZE      0x00100000
16
17  #define SHMEM_RESERVED_HEADERS 4K
18  #define SHMEM_RESERVED_SIZE (16384 * 1K)
19
20  MEMORY
21  {
22      shmem0 (rw) : ORIGIN = (SHMEM_START_ADDR + SHMEM_RESERVED_HEADERS),
23                  LENGTH = (SHMEM_RESERVED_SIZE - SHMEM_RESERVED_HEADERS)
24
25      SRAM2 (wx) : ORIGIN = (SRAM2_ADDR), LENGTH = SRAM2_SIZE
26  }

```

Figure 7: Section of linker_r5_hvl.ld showing the shmem reservation

Subsequently, the linker script forces the allocation of the VirtQueues strictly within this `shmem0` region

```
28 SECTIONS
29 {
30     shmem0 (NOLOAD): {
31         __shmem0_start = .;
32         KEEP(*(.shared.vring.*));
33         __shmem0_pool_start = .;
34         __shmem0_end = __shmem0_start + (SHMEM_RESERVED_SIZE - SHMEM_RESERVED_HEADERS);
35     } > shmem0
```

Figure 8: Section of `linker_r5_hvl.ld` showing the allocation of the VRings

The directive `KEEP(*(.shared.vring.*))`¹ is the exact C-level implementation of the **glue code** that places the VirtQueue control structures (the VRings) physically inside the shared memory. This ensures that when the Zephyr driver initializes the VirtQueues, the control rings are not placed in the standard SRAM2 memory block, but are exposed at the `0x37000000` address space, making them symmetrically accessible by the Linux master.

2.2 Inter-Processor Interrupts (IPI) vs. VM-Exits

As seen in the system architecture, the mechanism involving notification through a VirtIO doorbell register (so called “kick”) is physically impossible because both operating systems run natively on bare metal without a hypervisor to intercept memory accesses. Therefore, the VM-exit must be replaced by this physical hardware notification mechanism called **Inter-Processor Interrupt (IPI)**.

The necessary code responsible for this crucial architectural adaptation in the Zephyr firmware is located in the `src/xlnx_ipi.c` file, which acts as the **custom driver** for the Xilinx ZynqMP IPI hardware block. When the Zephyr application needs to notify the Linux master that new descriptors have been added to the Available Ring, the OpenAMP VirtIO-MMIO transport layer calls the hardware-specific notify function. In `xlnx_ipi.c`, this is explicitly implemented in the `xlnx_ipi_notify` function:

```
78 static void xlnx_ipi_notify(const struct device *dev)
79 {
80     WRITE32(dev, IPI_TRIG, (1 << 24));
81 }
```

Figure 9: Section of `xlnx_ipi.c` showing the notify function

The `WRITE32` macro performs a direct memory-mapped write to the `IPI_TRIG`^{**} hardware register. This exact line of code is the physical manifestation of the hypervisorless “kick”. By writing to this register, the Cortex-R5F core raises a **physical interrupt line** that is caught by the Linux master's `user-mbox.ko` kernel module, effectively waking up the PMM to process the network or entropy request.

Conversely, the system must also handle the reverse notification path (when the Linux device finishes processing the buffers and updates the Used Ring). Instead of the hypervisor injecting a virtual interrupt (vCPU IRQ) into the guest, the Zephyr firmware registers a **physical Interrupt Service Routine (ISR) to catch incoming IPIs** from Linux. This is found in the `xlnx_ipi_isr` function:

¹ *KEEP directive prevents linker optimization (garbage collection), forcing the VRing structures to be maintained in the memory, since Zephyr compiler is not aware of the external accesses performed by the Linux PMM*

^{**} Physical memory mapped register that belongs to a specific block of the Zynq UltraScale+ chip, called IPI Block (Inter-Processor Interrupt Block)

```

50 static void xlnx_ipi_isr(const struct device *dev)
51 {
52     uint32_t isr, imr;
53     uint32_t i;
54
55     isr = READ32(dev, IPI_ISR);
56     imr = READ32(dev, IPI_IMR);
57
58     for (i = 0 ; i < 32; i++)
59     {
60         if (imr & (1 << i)) /* masked */
61             continue;
62         if (!(isr & (1 << i))) /* not active */
63             continue;
64         /* ACK interrupt */
65         WRITE32(dev, IPI_ISR, (1 << i));
66     }
67
68 #ifdef HVL_VIRTIO
69     virtio_mmio_hvl_cb_run();
70 #endif

```

Figure 10: Section of xlnx_ipi.c showing the catching ISR

When the hardware interrupt fires, this ISR is executed. The `virtio_mmio_hvl_cb_run()` function acts as the callback that hooks back into the OpenAMP libraries, instructing the VirtIO driver to scan the Used Ring for newly returned payloads (such as the incoming ICMP echo replies for the ping command).

2.3 Bounce Buffers Implementation

In a traditional hardware-assisted virtualized system, memory management is dynamic. When the guest OS wants to transmit a network packet, the standard VirtIO driver simply places a pointer to its internal RAM buffer into the Descriptor Ring. The hypervisor (KVM), having full privileged access to the entire system memory, uses host-virtual to guest-physical address translation to read the payload directly from the guest's local RAM.

In the hypervisorless AMP architecture, this mechanism breaks down. The Linux master (executing the PMM) **cannot** arbitrarily or safely access Zephyr's local SRAM. If the Zephyr driver placed a raw internal SRAM pointer into the Descriptor Ring, the Linux master would fail to read the packet. This strict physical memory isolation necessitates the implementation of **bounce buffers**.

To bridge this gap without modifying the high-level standard VirtIO network and entropy drivers, the hypervisorless transport layer (`virtio_mmio_hvl`) acts as a transparent shim layer. It intercepts the buffer handling by overriding standard VirtIO memory allocation routines through **OS-dependent weak functions**.

When the Zephyr virt-net driver prepares an ICMP ping packet, the execution flow deviates from the standard VirtIO path:

1. **Allocation:** instead of using the packet's original memory address, the OS-dependent weak function intervenes and allocates a contiguous memory block from the shared memory pool.

This pool is strictly defined in the linker_r5_hvl.ld script (starting at the `__shmem0_pool_start` symbol within the `0x37000000` region).

2. **Explicit Copy:** once the bounce buffer is allocated in the shared memory, an explicit memory copy operation (`memcpy`, abstracted via libmetal I/O block write functions) is performed. The payload bytes of the ping packet are **physically duplicated** from Zephyr's private SRAM into the shared bounce buffer.
3. **Descriptor Update:** only after the payload is successfully copied does the driver update the Descriptor Ring. The `addr` field of the newly added descriptor is populated with the physical address of the bounce buffer (e.g., `0x37000xxx`), rather than the internal Zephyr memory pointer.

Finally, the hardware notification (IPI) is triggered via the `xlnx_ipi_notify` function. When the Linux `user-mbox` receives the interrupt and wakes up the PMM, the Linux master reads the Descriptor Ring, follows the pointer, and safely retrieves the payload data directly from the mutually accessible shared memory space. This design cleanly completely decouples the control plane (VRings) from the data plane, enabling hypervisorless communication at the cost of a deliberate memory-to-memory copy overhead.

2.4 End-to-End Execution Trace (virt-rng and virt-net)

To comprehensively address the tracking of a VirtIO request from start to finish, both an entropy read (`virt-rng`) and a network packet transmission (`virt-net`) can be traced.

Tracing a virt-rng Read:

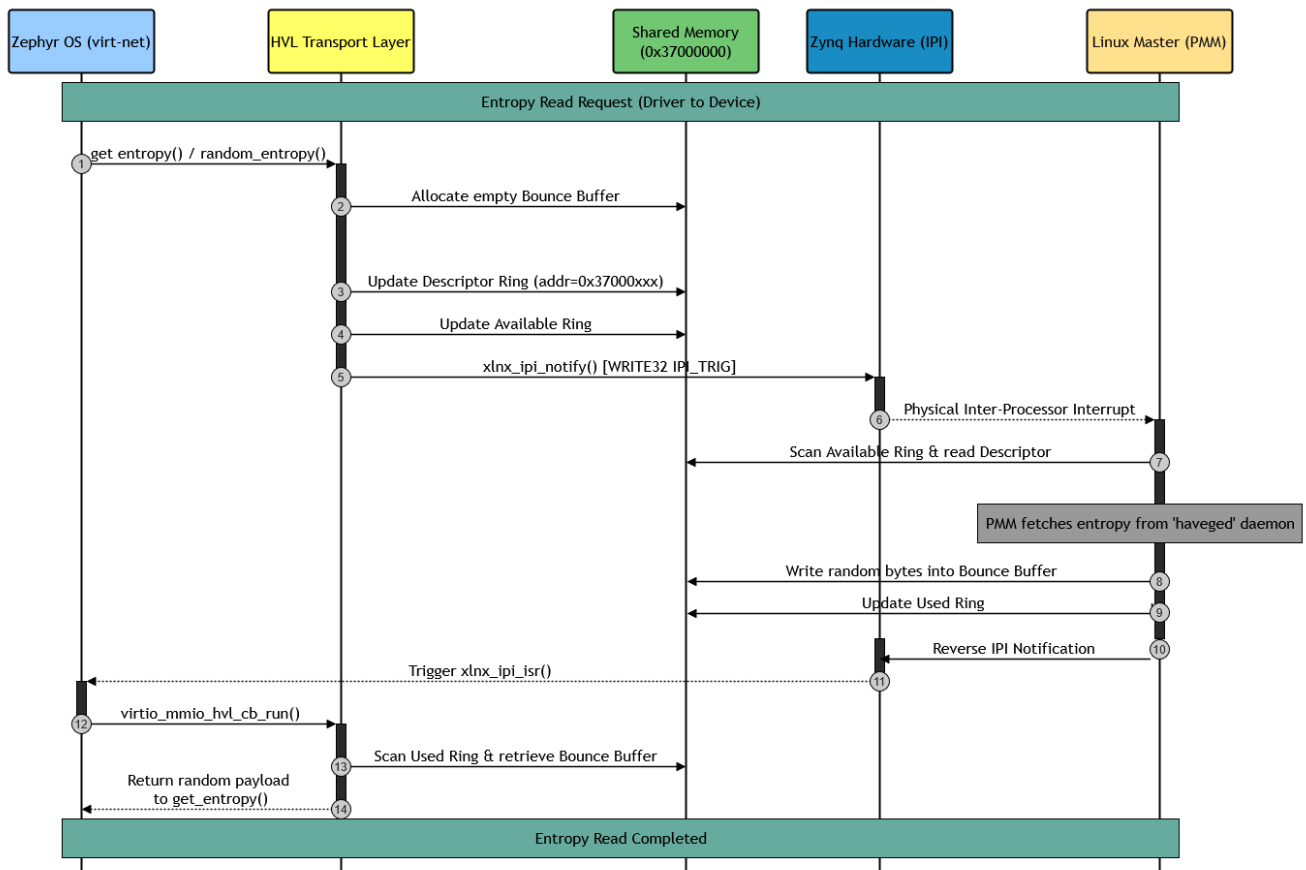


Figure 11: Low-level sequence diagram tracing a Virt-RNG entropy Read request.

The execution begins in the Zephyr application code, where the `get_entropy()` function is invoked. The OpenAMP shim layer allocates an empty bounce buffer in the shared memory and adds its descriptor to the Descriptor Ring. After the `xlnx_ipi_notify` hardware kick, the Linux PMM reads the descriptor, fetches the random bytes generated by the haveged daemon, and writes them directly into the bounce buffer. A reverse IPI wakes up Zephyr's `xlnx_ipi_isr`, allowing the driver to safely consume the random payload.

Tracing a virt-net Packet (Ping)

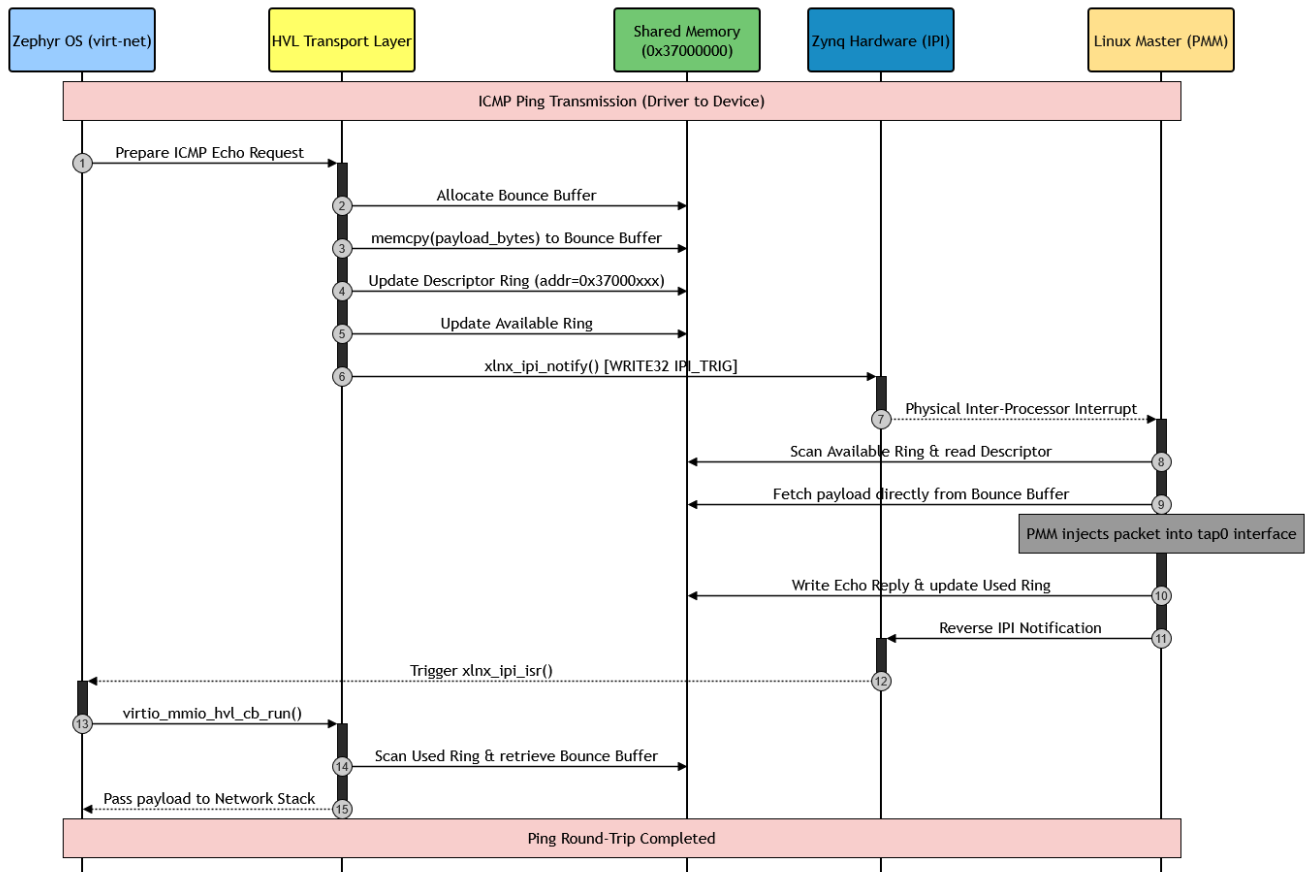


Figure 12: Low-level sequence diagram tracing a Virt-Net ICMP Ping request

The flow for the ICMP ping is identical in its control mechanics but differs in the **payload direction**. When the Zephyr network stack sends an ICMP Echo Request, the transport layer intercepts the raw internal SRAM pointers. It explicitly allocates a bounce buffer in the `0x37000000` shared memory region and performs a strict memory-to-memory copy (*memcpy*) of the payload into it. After updating the Descriptor Ring and triggering the `IPI_TRIG` register, the Linux PMM fetches the payload directly from the shared memory and injects the ping into the `tap0` virtual interface.

3 Experiments and Latency Observations

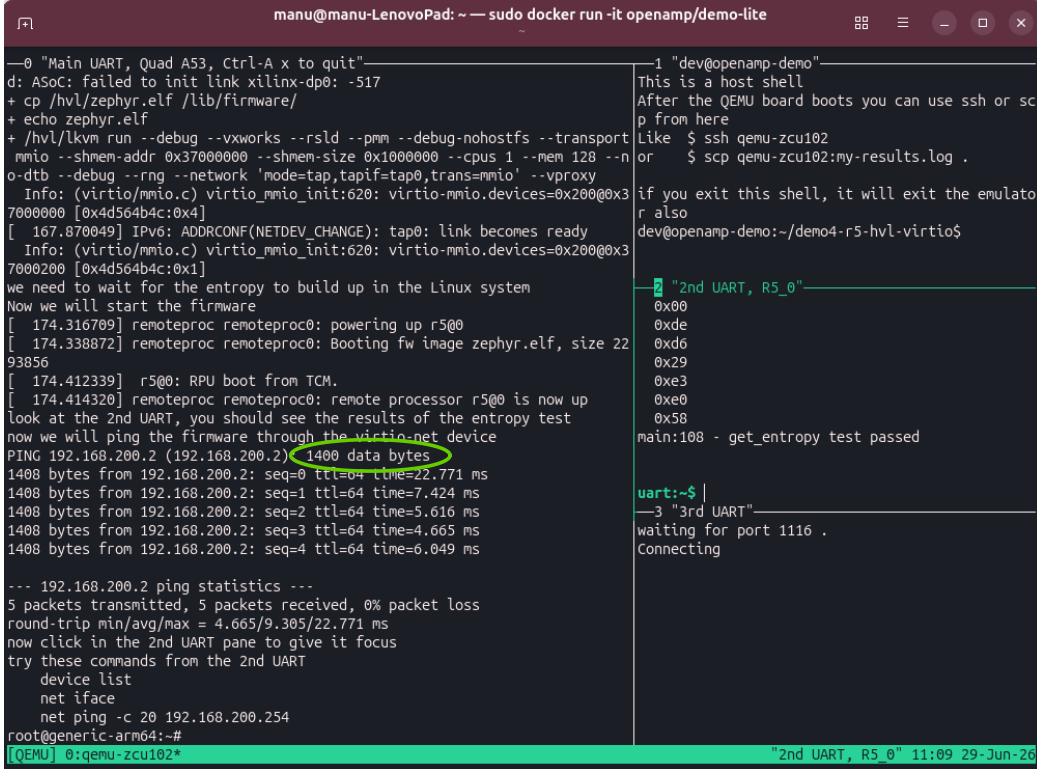
Moving beyond the static source-code analysis, it is of major interest to introduce practical experiments conducted within the QEMU-emulated Zynq UltraScale+ environment. The objective is to apply controlled modifications to the baseline execution flow to validate the system's behavior, inspect the shared memory dynamically, and record approximated Inter-Processor Communication (IPC) timing.

3.1 Custom Modifications

3.1.1 Network payload modification

Custom modifications were introduced to the standard execution environment on both the Linux master and the Zephyr remote.

On the Linux side, the network testing parameters within the **demo4** script were intentionally altered. The default ping command was modified to include a significantly larger data payload using the **-s** flag.



```
manu@manu-LenovoPad: ~ -- sudo docker run -it openamp/demo-lite
--0 "Main UART, Quad A53, Ctrl-A x to quit"
d: ASoC: failed to init link xilinx-dp0: -517
+ cp /hvl/zephyr.elf /lib/firmware/
+ echo zephyr.elf
+ /hvl/lkvm run --debug --vxworks --rslid --pmm --debug-nohostfs --transport
mmio --shmem-addr 0x37000000 --shmem-size 0x1000000 --cpus 1 --mem 128 --n
o-dtb --debug --rng --network 'mode=tap,tapif=tap0,trans=mmio' --vproxy
Info: (virtio/mmio.c) virtio_mmio_init:620: virtio-mmio.devices=0x200@0x3
7000000 [0x4d564b4c:0x4]
[ 167.870049] IPv6: ADDRCONF(NETDEV_CHANGE): tap0: link becomes ready
Info: (virtio/mmio.c) virtio_mmio_init:620: virtio-mmio.devices=0x200@0x3
7000200 [0x4d564b4c:0x1]
we need to wait for the entropy to build up in the Linux system
Now we will start the firmware
[ 174.316709] remoteproc remoteproc0: powering up r5@0
[ 174.338872] remoteproc remoteproc0: Booting fw image zephyr.elf, size 22
93856
[ 174.412339] r5@0: RPU boot from TCM.
[ 174.414320] remoteproc remoteproc0: remote processor r5@0 is now up
look at the 2nd UART, you should see the results of the entropy test
now we will ping the firmware through the virtio-net device
PING 192.168.200.2 (192.168.200.2) 1400 data bytes
1408 bytes from 192.168.200.2: seq=0 ttl=64 time=22.771 ms
1408 bytes from 192.168.200.2: seq=1 ttl=64 time=7.424 ms
1408 bytes from 192.168.200.2: seq=2 ttl=64 time=5.616 ms
1408 bytes from 192.168.200.2: seq=3 ttl=64 time=4.665 ms
1408 bytes from 192.168.200.2: seq=4 ttl=64 time=6.049 ms

--- 192.168.200.2 ping statistics ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max = 4.665/9.305/22.771 ms
now click in the 2nd UART pane to give it focus
try these commands from the 2nd UART
device list
net iface
net ping -c 20 192.168.200.254
root@generic-arm64:~#
[QEMU] 0:qemu-zcu102*
1 "dev@openamp-demo"
This is a host shell
After the QEMU board boots you can use ssh or sc
p from here
Like $ ssh qemu-zcu102
or $ scp qemu-zcu102:my-results.log .
if you exit this shell, it will exit the emulato
r also
dev@openamp-demo:~/demo4-r5-hvl-virtio$
2 "2nd UART, R5_0"
0x00
0xde
0xd6
0x29
0xe3
0xe0
0x58
main:108 - get_entropy test passed
uart:~$ |
3 "3rd UART"
waiting for port 1116 .
Connecting
"2nd UART, R5_0" 11:09 29-Jun-26
```

Figure 13: Ping performed with a bigger size payload

This modification explicitly stresses the virt-net bounce buffers located in the shared memory, forcing the OS-dependent weak functions to allocate and perform **memcpy** operations on larger packets. The Zephyr remote successfully caught the corresponding Inter-Processor Interrupts (IPIs) and responded to the enlarged ICMP Echo Requests without fragmentation nor errors, demonstrating the robustness of the transparent bounce layer.

3.1.2 Static Memory Inspection

It is remarkable to prove the static initialization of the hypervisorless VirtIO devices within the pre-shared memory pool. As defined in the custom linker script (`linker_r5_hvl.ld`), the memory-mapped I/O (MMIO) transport relies on a shared memory region starting at the physical base address `0x37000000`.

Standard user-space tools on the Linux master (like `devmem` or `hexdump`) were correctly blocked by the kernel's `CONFIG_STRICT_DEVMEM` security protection, which prevents direct `/dev/mem` access to regions actively managed by the PMM. To bypass the OS restrictions, the hardware emulator's out-of-band console (**QEMU Monitor**) was leveraged using the **xp** (*examine physical*) command.

```
(qemu) xp /32xw 0x37000000
0000000037000000: 0x74726976 0x00000001 0x00000004 0x4d564b4c
0000000037000010: 0x00000000 0x00000000 0x00000000 0x00000000
0000000037000020: 0x00000000 0x00000000 0x00001000 0x00000000
0000000037000030: 0x00000000 0x00000030 0x00000002 0x00001000
0000000037000040: 0xaabbaabb 0x00000000 0x00000000 0x00000000
0000000037000050: 0x00000000 0x00000000 0x00000000 0x00000000
0000000037000060: 0x00000000 0x00000000 0x00000000 0x00000000
0000000037000070: 0x00000004 0x00000000 0x00000000 0x00000000
(qemu) |
[QEMU] 0:qemu-zcu102*
```

Figure 14: Memory dump at shared memory base address

The hexadecimal output at the base address can be directly decoded (following the Little-Endian format) to expose the following VirtIO parameters:

- **0x74726976**: in ASCII, this translates to the "**v i r t**" magic string, which is the standard signature indicating a valid VirtIO device header.
- **0x00000001**: indicates the VirtIO device version (Version 1).
- **0x00000004**: represents the Device ID. According to the VirtIO specification, ID 4 corresponds exactly to the **Virt-RNG** (entropy) device.
- **0x4d564b4c**: in ASCII, this translates to "**L K V M**", the Vendor ID. This provides irrefutable proof that the device was not created by a hardware hypervisor, but rather by the user-space lkvm PMM tool running on Linux.
- **0xaabbaabb**: found at offset 0x40, this value perfectly matches the HVL_CFG_ACK macro defined in Zephyr's `xlnx_ipi.c` driver, acting as a custom acknowledgment flag for the hypervisorless configuration.

This physical snapshot proves that the user-space lkvm tool bypasses the need for a hypervisor by statically injecting native VirtIO structures directly into the pre-shared memory domain.

3.1.3 Dynamic Payload interception via Descriptor Rings

To provide empirical evidence of the actual shared memory utilization, a **deep dynamic memory inspection** experiment was performed. The objective was to intercept the network payload physically traversing the shared memory bounce buffers, proving that the user-space PMM handles data without a traditional hypervisor.

- **Step 1 - Generating network traffic with custom payload**: on the Linux master, the standard `ping` command was customized to flood the Zephyr remote with packets containing a specific payload. Since the minimal BusyBox `ping` command restricts the pattern to a single byte, the hexadecimal value **52** (corresponding to the ASCII letter '**R**', for Real-Time) was chosen. The command was executed in the background to keep the terminal free from output clutter:
`ping -p 52 -s 128 -c 500 192.168.200.2 > /dev/null &`
- **Step 2 - Accessing memory via QEMU Monitor**: while the `ping` was continuously pumping packets into the network, the execution was intercepted by entering the multiplexed QEMU Monitor console. This provided superior read access to the physical memory of the emulated system without triggering kernel security policies.
- **Step 3 - Dynamic descriptor ring traversal**: instead of blindly scanning the 16MB shared memory region (`shmem0`), the **VirtIO Descriptor Ring** was manually inspected. Knowing from the linker script that the VRings are allocated immediately after the 4KB header reservation (around `0x37001000`), the physical memory was dumped using the `xp /256xw` command.

```

root@generic-arm64:~# ping -p 52 -s 128 -c 500 192.168.200.2 > /dev/nul
l &
root@generic-arm64:~# QEMU 5.1.0 monitor - type 'help' for more informa
tion
(qemu) xp /256xw 0x37001000
0000000037001000: 0x370078e8 0x00000000 0x000000b4 0x00010000
0000000037001010: 0x00000000 0x00000000 0x00000000 0x00020000
0000000037001020: 0x00000000 0x00000000 0x00000000 0x00030000
0000000037001030: 0x00000000 0x00000000 0x00000000 0x80000000
0000000037001040: 0x00290000 0x00000000 0x00000000 0x00000000
0000000037001050: 0x00000000 0x00000000 0x00000000 0x00000000
0000000037001060: 0x00000000 0x00000000 0x00000000 0x00000000
0000000037001070: 0x00000000 0x00000000 0x00000000 0x00000000
0000000037001080: 0x00000000 0x00000000 0x00000000 0x00000000
0000000037001090: 0x00000000 0x00000000 0x00000000 0x00000000

```

Figure 15: Exploring shared memory to find Descriptor Ring

By traversing the Descriptor Ring, it was possible to **dynamically locate the physical pointer to the active Bounce Buffer** currently allocated by the libmetal library (in this specific instance, resolving to `0x370078e8`).

- **Step 4 - Unmasking payload inside the buffer:** having acquired the exact physical address of the Bounce Buffer, a final memory dump was performed, forcing the QEMU Monitor to decode the bytes as ASCII characters using the following command:

```
xp /128c 0x370078e8
```

The “c” allows to decode memory in a character format for a better readability.

```

(qemu) xp /128c 0x370078e8
00000000370078e8: '\x19' '\x03' '\x00' '\x00' '\x19' '\x03' '\x00' '\x0
0' '\x00' '\x00' '\x8a' '9' '\x05' '\xa7' 'o' '\x08'
00000000370078f8: '\x00' '\x00' '\x00' '\x00' '\x00' '\x00' '\x00' '\x0
0' 'E' '\x00' '\x00' '\x9c' '\x00' '\x00' '\x00' '\x00'
0000000037007908: '@' '\x01' 'h' '\x0f' '\xc0' '\xa8' '\xc8' '\x02' '\x
c0' '\xa8' '\xc8' '\xfe' '\x00' '\x00' 'O' '\x90'
0000000037007918: '\x02' 'U' '\x00' 'l' 'v' '\xb3' 'G' '\x0b' 'R' 'R' '
R' 'R' 'R' 'R' 'R' 'R'
0000000037007928: 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' '
R' 'R' 'R'
0000000037007938: 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' '
R' 'R' 'R'
0000000037007948: 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' '
R' 'R' 'R'
0000000037007958: 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' '
R' 'R' 'R'
0000000037007968: 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' '
R' 'R' 'R'
0000000037007978: 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' '
R' 'R' 'R'
0000000037007988: 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' '
R' 'R' 'R'
0000000037007998: 'R' 'R' 'R' 'R' '\x00' '\x00' '\x00' '\x00' '\x18' '\
x00' '\x00' '\x00' '\x96' '\xe1' '?' '\x00'

```

Figure 16: Custom payload found in Bounce Buffer

As clearly visible in the [Figure 15](#) memory dump, the first portion of the buffer contains the non-printable routing bytes corresponding to the MAC, IP, and ICMP headers generated by the virt-net driver. Immediately following the headers, the custom payload emerges flawlessly: a continuous block of 'R' characters in cleartext.

3.1.4 Custom Instrumentation and VirtIO-RNG Observation

To validate the underlying hypervisorless VirtIO architecture and demonstrate control over the data plane, a **custom instrumentation** was introduced into the Zephyr RTOS firmware*.

The experiment focused on modifying the behavior of the `virt-rng` (Random Number Generator) frontend driver to **expose the raw payload data exchanged with the Linux backend**.

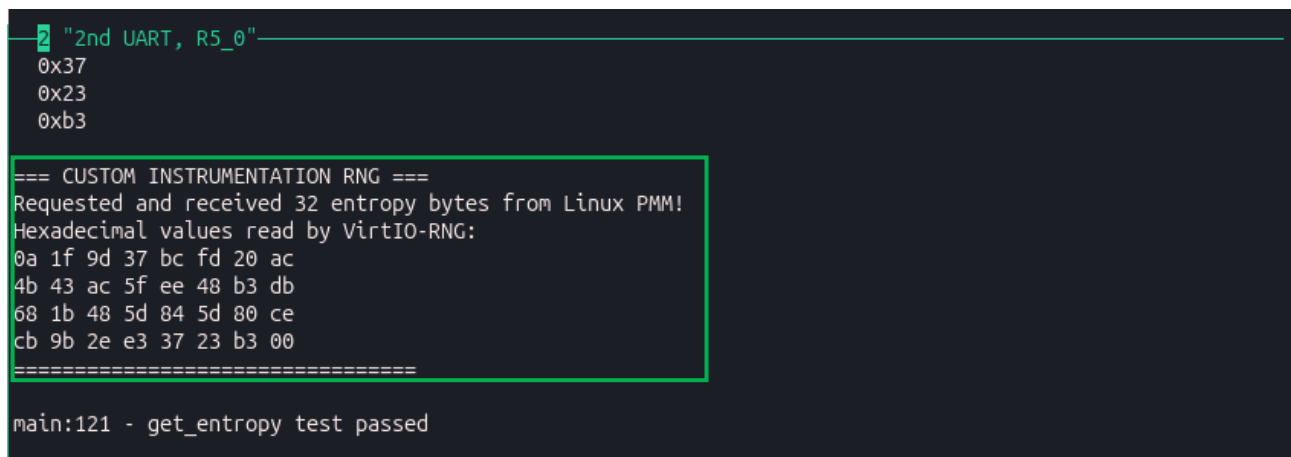
In the original implementation, the Zephyr firmware silently performs a basic entropy test during the boot sequence: it requests a small buffer of 10 bytes from the VirtIO-RNG device, verifies that the buffer is not filled with default zero values, and simply outputs a *get_entropy test passed* message upon success.

To deeply analyze the communication, the Zephyr `get_entropy()` routine was **modified** to request a larger payload of **32 bytes** of entropy and to explicitly dump the received data to the UART console.

This modification successfully triggered the complete hypervisorless execution flow:

1. The Zephyr `virt-rng` driver allocated an empty bounce buffer within the usual shared memory region and placed its pointer into the Descriptor Ring.
2. Zephyr side raised a physical Inter-Processor Interrupt (IPI) to notify the Linux master.
3. The Linux PMM (lkvm), running in user-space, intercepted the request and fetched the requested 32 bytes of high-quality randomness generated by the `haveged` daemon on the Linux host.
4. The PMM wrote the 32 bytes directly into the shared bounce buffer, updated the Used Ring, and sent a reverse IPI to Zephyr.

As captured during the execution, the Zephyr console successfully displayed the custom instrumentation output



```
"2nd UART, R5_0"
0x37
0x23
0xb3

=== CUSTOM INSTRUMENTATION RNG ===
Requested and received 32 entropy bytes from Linux PMM!
Hexadecimal values read by VirtIO-RNG:
0a 1f 9d 37 bc fd 20 ac
4b 43 ac 5f ee 48 b3 db
68 1b 48 5d 84 5d 80 ce
cb 9b 2e e3 37 23 b3 00
=====
main:121 - get_entropy test passed
```

Figure 17: Zephyr print after custom instrumentation

Upon receiving the reverse IPI, the Zephyr callback (`virtio_mmio_hvl_cb_run`) successfully retrieved the bounce buffer. Here, the custom instrumentation dumped the **raw hexadecimal bytes** to the

* The cross-compilation of the custom `zephyr.elf` firmware required a specific integration process to overcome dependency limitations. The standard Zephyr package manager (**west**) defaulted to the stable OpenAMP release, which lacked the experimental hypervisorless extensions. There was therefore the need to force the compiler to exclusively link the experimental frontend modules in order to resolve all dependency conflicts.

secondary UART console, proving the successful end-to-end lateral communication with the injected custom behaviour.

3.2 IPC Timing Measurements

To evaluate the latency introduced by the hypervisorless VirtIO architecture, approximate timing measurements were recorded along the Inter-Processor Communication (IPC) path. According to the literature, baseline IPC latencies on physical bare-metal AMP systems can be as low as a few microseconds.

In this experiment, the end-to-end communication delay was measured by analyzing the **Round-Trip Time (RTT)** of the ICMP Echo Requests (ping) exchanged between the Linux master (APU) and the Zephyr remote (RPU) over the VirtIO-MMIO transport.

As captured during the default bidirectional testing phase, the transmission of standard 56-byte data payloads yielded the following latency statistics:

- **Minimum latency:** *~4.47 ms*
- **Average latency:** *~9.76 ms*
- **Maximum latency:** *~20.31 ms*

These measurements encompass the entire hypervisorless stack overhead: the initial memory-to-memory copy into the shared bounce buffers, the triggering of the Inter-Processor Interrupts (IPIs) via the *IPI_TRIG* hardware register, the Linux `user-mbox` context switch, and the final payload retrieval by the respective OS network stacks.

3.2.1 Influence of Payload Size

To further investigate the data plane behavior, data was also retrieved from the additional test conducted by forcefully increasing the network payload. As illustrated in [Figure 13](#), the ICMP Echo Request size was expanded from the default 56 bytes to **1400 bytes** (near the MTU limit).

The statistics for this enlarged payload showed an increased maximum latency of *~22.77 ms*. This variation highlights two critical aspects of the architecture; moving 1400 bytes requires **longer explicit memory-to-memory copies (memcpy)** into the shared static region compared to standard packets, which intrinsically increases the overall processing time before the IPI is fired.

3.3 Limitations of Emulated Environments

It is mandatory to explicitly state that the timing measurements obtained in this experiment **are not representative of absolute real-time hardware performance**.

The execution environment relies entirely on QEMU, a **software** emulator. Unlike physical execution on a real Zynq UltraScale+ MPSoC (where real-time tasks on the Cortex-R5 processor are highly deterministic), the emulated processors are mapped to standard threads on the host PC's operating system (in this case Ubuntu). Consequently, the recorded ping latencies (in the millisecond range) are heavily distorted by the host OS scheduler, background processes, and the virtualization overhead of the Docker container.

Therefore, these results successfully demonstrate the functional behavior and the relative trends of the hypervisorless VirtIO architecture, but they cannot be used to assess strict adherence to **industrial real-time deadlines**. A prime example of such stringent constraints is found in **Smart Grid** applications. In modern automated electrical substations, it is mandatory to guarantee an end-to-end

latency of less than 3 ms when generating, transmitting, and processing critical layer-2 messages like **Generic Object-Oriented Substation Events (GOOSEs)** and Sampled Values (SV). Since the emulation overhead pushes our recorded latencies beyond this threshold, accurate performance profiling and jitter evaluation to prove compliance with such industrial standards would strictly require porting these artifacts to a physical board.

4 Synthesis and Analysis

4.1 VirtIO Utilization

In the hypervisorless architecture demonstrated, the system utilizes the standard VirtIO-MMIO (Memory-Mapped I/O) transport. Regarding device coverage, the Zephyr RTOS successfully exercises the `virt-net` (network) and `virt-rng` (entropy generation) frontend drivers. However, because there is no hypervisor to orchestrate the environment, several dynamic parts of the standard VirtIO specification are stubbed or completely bypassed:

- **Feature Negotiation:** in a standard KVM environment, the hypervisor and guest negotiate device features dynamically. Here, feature negotiation is replaced by predefined feature lists statically defined by the system architect, that are placed in the `virtio` device header data structure directly placed in the fixed shared memory as seen before.
- **Dynamic Memory Allocation:** in standard VirtIO, the guest driver decides where to allocate memory for `VirtQueues` and data buffers, using allocation functions that simply ask for dynamic RAM memory. In the hypervisorless paradigm, these buffer allocation routines are overridden via **custom OS-dependent weak functions** that force the allocation strictly within a statically pre-defined shared memory region, in a completely transparent way to the high-level drivers `virt-net` and `virt-rng`.

4.2 The Hypervisorless Paradigm

The goal of the hypervisor-less VirtIO initiative is to prototype and define a framework for using VirtIO as a communication infrastructure, while removing the penalties associated with transitions from guest to hypervisor and back. And, in the process, enable VirtIO to be used for heterogeneous multicore devices which leverage compute islands rather than virtualization for hardware partitioning.

What makes this paradigm truly "hypervisor-less" is the complete elimination of a heavyweight Virtual Machine Monitor (VMM) like KVM, operating without hardware CPU virtualization extensions. Instead, it relies on this lightweight user-space **Physical Machine Monitor (PMM)** running on Linux via the `lkvm` tool. To achieve this, fundamental VirtIO assumptions had to be radically adapted:

- **VM-Exits replaced by IPIs:** standard `virtio` relies on VM-exits to trap hardware accesses and notify the hypervisor. Without a hypervisor, notifications are handled by physical hardware Inter-Processor Interrupts (IPIs) triggered by writing directly to hardware registers (e.g., `IPI_TRIG`).
- **Bounce Buffers:** since the Linux PMM cannot dynamically translate guest-physical addresses into host-virtual addresses to read Zephyr's internal SRAM, the system uses pre-allocated "bounce buffers" in the shared memory. Descriptors point to these buffers, requiring an explicit memory-to-memory copy (`memcpy`) of the payload.

4.3 VirtIO vs. RPMsg

Traditionally, the OpenAMP framework facilitates Inter-Processor Communication (IPC) between heterogeneous cores using the **RPMsg** protocol. RPMsg is an established standard, but it operates purely as a **low-level transport layer**: it provides channels and endpoints to exchange "raw" arrays of bytes, but it has no knowledge of what those bytes represent. Consequently, if a developer wants to share complex resources (like a network interface or a file system) over RPMsg, they are forced to write custom, proprietary Remote Procedure Call (RPC) frameworks on top of it to encode and decode the data. This leads to a massive waste of time and reinventing the wheel.

VirtIO solves this by elevating the communication from raw byte channels to a **standardized device abstraction layer**. VirtIO defines precise, universally recognized device specifications, like `virt-net` for networking and `virt-rng` for entropy as previously seen, but originally exclusive for virtualization purposes and therefore unused out of that domain. Moreover, both the Linux kernel and the Zephyr RTOS already possess mature, standard drivers for these devices. By using VirtIO **directly** over shared memory, elevating it at a higher level, the Zephyr RTOS interacts with the Linux backend exactly as if it were interacting with **physical hardware peripherals**. This unlocks massive software reuse: developers do not need to write custom RPC protocols but can "*plug and play*" standard VirtIO drivers. The **cost** is to be found in performance **overhead** and rigidity; RPMsg can be highly optimized for zero-copy transfers, whereas the hypervisorless VirtIO adaptation mandates strict static memory layouts and relies on explicit data copies (`memcpy`) into the intermediate bounce buffers, inherently adding latency to the communication path.

4.4 Current Limitations

While promising, the current hypervisorless VirtIO implementation has notable limitations in terms of **security**, since the backend device (PMM) operates in Linux user-space and has direct access to the pre-shared memory pool. In systems lacking strict hardware separation, this shared access could expose the RTOS memory to vulnerabilities originating from a rich OS*.

Performance overhead is another aspect. Unlike custom RPMsg applications that are **intrinsically aware of the shared memory** layout and can achieve true zero-copy transfers, standard VirtIO frontend drivers lack this awareness and naturally allocate packets in the RTOS's private SRAM. Consequently, the architectural "shim layer" is forced to explicitly copy these payloads into the shared bounce buffers to allow communication, which inherently limits network throughput and increases processing latency.

Lastly, **flexibility** and **maturity** are arguably lacking, because the solution relies on static, hardcoded configurations (e.g., linker scripts mapping exact physical addresses like `0x37000000`) rather than dynamic discovery. Furthermore, it heavily relies on custom glue-code and resides in an experimental branch (`virtio-exp`), lacking full device coverage.

4.5 Real-Time Relevance

To honestly evaluate the claim that this architecture "*removes real-time limitations*", there's the need to distinguish between the RTOS execution and the communication path. As previously observed, traditional hypervisors (like **Xen**) introduce massive worst-case latencies (failing the aforementioned $< 3\text{ms}$ industrial constraints for Smart Grid GOOSE/SV messages, reaching peaks of over 100 ms under stress) due to VM-exits, timer emulation, and hypervisor scheduling overhead. Removing the hypervisor **truly helps** the real-time core (Cortex-R5): it runs on bare-metal with microsecond-level determinism, responding to direct hardware IPIs without any virtualization jitter. However, the

* *General-Purpose* operating system, generally completer and more complex, being therefore incredibly exposed to vulnerabilities or hacker attacks.

communication path **does add some latency**: the explicit **memcpy** into the bounce buffers and the reliance on the Linux OS scheduler (via the `user-mbox` driver) to wake up the PMM introduces latency in the millisecond range. Therefore, while the RTOS regains absolute determinism for its local tasks, the inter-processor data exchange still suffers from the jitter typical of general-purpose operating systems.

4.6 Future Developments

The custom shim layers and bounce-buffer workarounds explored in this experimental demo are paving the way for a more formalized standard. The trajectory of this technology is evolving towards a generalized transport protocol named **virtio-msg**, specifically its variant **virtio-msg-amp**. This evolution is being actively driven by Linaro's **HVAC (Heterogeneous VirtIO for Automotive Computing)** project. The ultimate goal is to standardize the interactions between main application cores and coprocessors without hypervisors, standardizing the device sharing for modern mixed-criticality industrial and automotive systems, ensuring that VirtIO can be securely and deterministically used in next-generation AMP architectures.

5 (Optional) Real Hardware Execution on ZCU102

5.1 Hardware Setup and Execution

Porting the QEMU artifacts to the physical board

5.2 Absolute Latency Comparison

Real timing numbers comparing classic RPMsg against the hypervisorless virtio network path